

Lab06

Cá nhân hóa theo MSSV

|    |         |    |   |
|----|---------|----|---|
| s  | 2212453 | g  | 3 |
| d1 | 2       | k0 | 3 |
| d2 | 2       |    |   |
| d3 | 1       |    |   |
| d4 | 2       |    |   |
| d5 | 4       |    |   |
| d6 | 5       |    |   |
| d7 | 3       |    |   |

PHẦN 1. THỰC HÀNH EXCEL THEO BẢNG MẪU

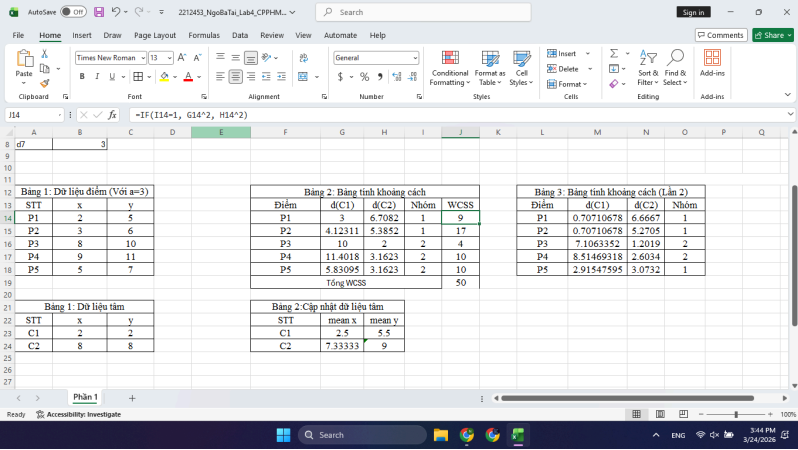
Vì sao thuật toán thích những tâm “ở giữa” cụm ? (Câu 6 phần hướng dẫn thực hiện theo bước)

Mục tiêu tối thượng của thuật toán k-Means là tìm cách làm cho tổng WCSS này càng nhỏ càng tốt. Khi tính bình phương, một khoảng cách lớn sẽ bị khuếch đại lên rất nhiều lần (ví dụ khoảng cách 5 thì bình phương là 25, nhưng khoảng cách 10 thì bình phương lên tới 100). Do đó, thuật toán rất “sợ” việc có một điểm nằm quá xa tâm. Để trung hòa và làm giảm mức phạt bình phương này, vị trí an toàn nhất, tối ưu nhất của tâm cụm bắt buộc phải là điểm trung bình cộng (ở giữa) của tất cả các điểm trong cụm. Bất kỳ sự lệch tâm nào sang một bên cũng sẽ làm khoảng cách tới các điểm ở bờ bên kia tăng vọt, kéo theo tổng WCSS tăng lên

Yêu cầu phải trả lời

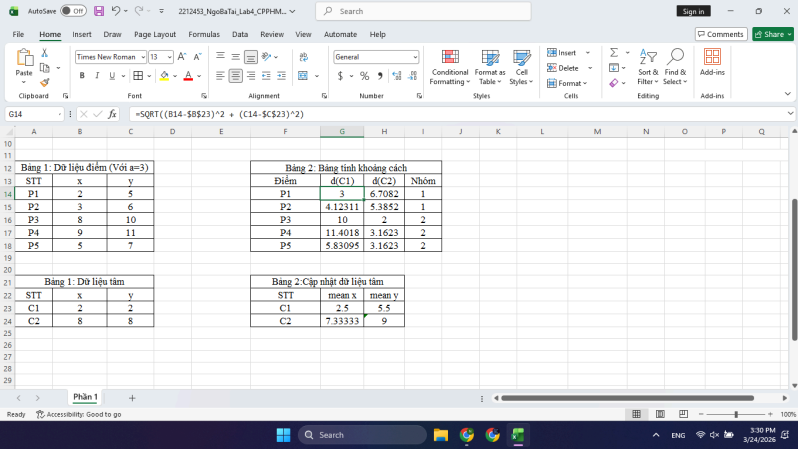
1. Điểm nào nằm lưng chừng giữa hai cụm và khó gán nhất.
- Điểm P2(3; 6) là điểm nằm lưng chừng và khó gán nhất. Nguyên nhân là khoảng cách từ P2 đến C1 (khoảng 4.12) và đến C2 (khoảng 5.39) có độ chênh lệch nhỏ nhất so với các điểm khác. Điều này cho thấy P2 nằm rất sát “đường biên giới” phân chia giữa hai cụm.
2. Sau một vòng lặp, tâm dịch chuyển theo hướng nào và vì sao.
- Sau một vòng lặp, tâm C1 dịch từ (2; 2) lên (2.5; 5.5), trong khi C2 dịch từ (8; 8) sang trái và lên trên thành (7.33; 9). Tâm luôn dịch chuyển thẳng về vị trí trung bình cộng (trọng tâm) của các điểm vừa được gán cho nó. Thuật toán làm vậy vì nó “thích” những tâm nằm ở giữa cụm để tối thiểu hóa tổng bình phương khoảng cách (WCSS).
3. Nếu thêm một điểm rất xa, tâm cụm có bị kéo lệch mạnh không.
- Nếu thêm một điểm rất xa, tâm cụm chắc chắn sẽ bị kéo lệch cực kỳ mạnh. Vì K-Means cập nhật tâm dựa trên phép tính trung bình cộng, nên nó đặc biệt nhạy cảm với các điểm ngoại lai (outlier). Giá trị tọa độ quá lớn của điểm này sẽ thao túng và làm sai lệch vị trí tâm mới của toàn bộ cụm

Bài tập làm thêm

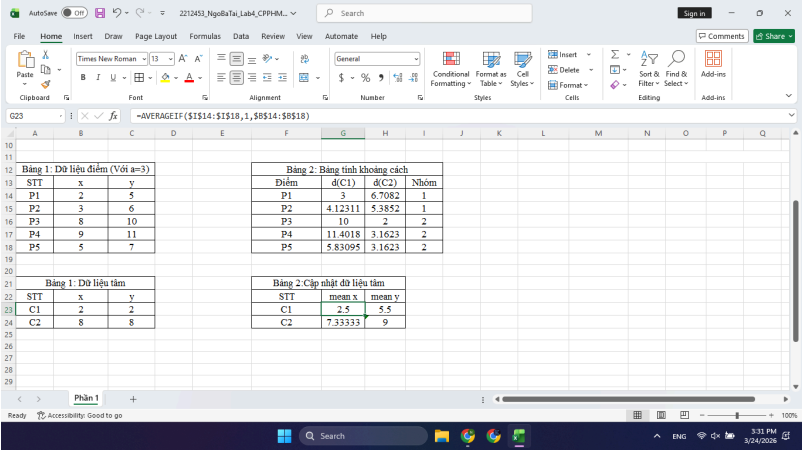


Minh chứng

• 01 ảnh toàn màn hình Excel có Formula Bar hiển thị công thức khoảng cách Euclid chứa địa chỉ tuyệt đối.



01 ảnh toàn màn hình Excel có Formula Bar hiển thị công thức hàm cập nhật tâm (AVERAGEIF).



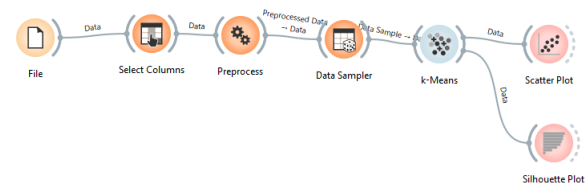
4–6 dòng giải thích về điểm khó gán cụm và sự dịch chuyển của tâm.

Điểm P2(3; 6) là điểm khó gán cụm nhất do nằm lưng chừng ở vùng ranh giới, với khoảng cách đến tâm C1 (4.12) và tâm C2 (5.39) có độ chênh lệch thấp nhất. Sau vòng lặp đầu tiên, các tâm khởi tạo đã dịch chuyển trực tiếp về hướng trọng tâm của các điểm vừa được phân vào nhóm. Cơ chế cập nhật này hoạt động bằng cách lấy trung bình cộng tọa độ của các thành viên để tạo ra tọa độ tâm mới. Sự dịch chuyển này đảm bảo tâm luôn nằm ở vị trí "ở giữa" cụm, qua đó giảm thiểu tối đa tổng bình phương khoảng cách (WCSS) từ các điểm dữ liệu đến tâm.

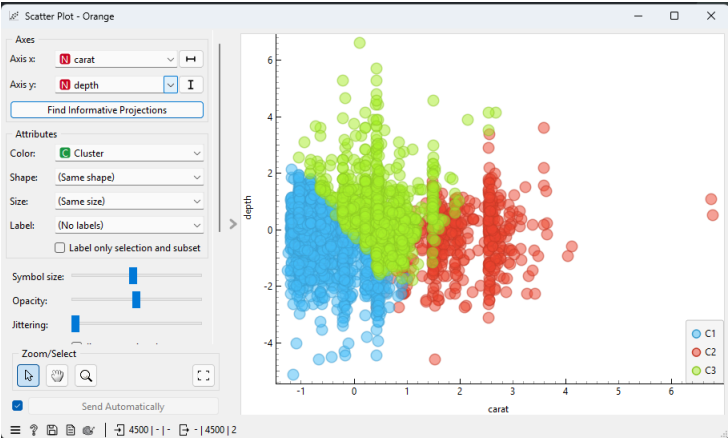
## PHẦN 2. ORANGE DATA MINING TRÊN REAL DATASET

### Mình chứng

- 01 ảnh mạch lạc Workflow Orange đầy đủ



- 01 ảnh Silhouette Plot hoặc Scatter Plot có màu cụm rõ ràng từ k-Means



- 3–5 dòng nhận xét về độ hợp lý của k0.

Mặc dù  $k_0 = 3$  giúp chia tách dữ liệu kim cương thành 3 nhóm cơ bản, nhưng độ hợp lý về mặt thuật toán chỉ ở mức tương đối do các điểm dữ liệu liên tục vẫn còn chồng lấn nhiều ở vùng ranh giới. Tuy nhiên, xét dưới góc độ lĩnh vực Thương mại điện tử, cấu trúc 3 cụm này lại cực kỳ phù hợp và có ý nghĩa thực tiễn cao. Việc phân rã tự động tạo ra 3 phân khúc sản phẩm rõ rệt (phổ thông, tầm trung và cao cấp) giúp các nền tảng bán lẻ trực tuyến dễ dàng quản lý kho hàng. Từ đó, doanh nghiệp có thể xây dựng các hệ thống gợi ý sản phẩm (recommendation) chính xác và chiến lược giá cả nhân hóa, giúp tiếp cận đúng nhu cầu của từng tệp khách hàng.

### Các yêu cầu phải trả lời

1. Với  $k_0 = 3$ , cụm có tách rõ hay còn chồng lấn nhiều?

Trả lời: Quan sát trên Scatter Plot (đặc biệt khi chiếu trên không gian 2 chiều carat và depth), các cụm có xu hướng phân tách theo chiều ngang (trục carat). Tuy nhiên, ở các vùng ranh giới (ví dụ giao điểm giữa cụm màu xanh dương và xanh lá, hoặc giữa xanh lá và đỏ), dữ liệu vẫn còn chồng lấn nhau khá nhiều. Điều này phản ánh việc K-Means đang cố gắng dùng các mặt phẳng tuyến tính để cắt một khối dữ liệu liên tục ra làm 3 phần, dẫn đến hiện tượng nhiễu ở biên.

2. Nếu Silhouette thấp, đó là do dữ liệu không rõ cụm hay do chọn sai số cụm?

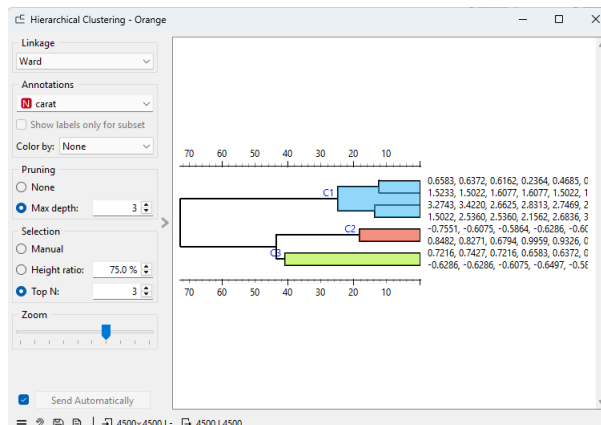
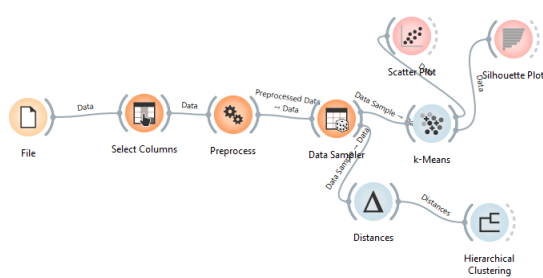
Trả lời: Điểm Silhouette thấp trong trường hợp này bắt nguồn từ cả hai nguyên nhân. Thứ nhất là do chọn sai số cụm (như đã phân tích ở phần Python,  $k=4$  cho điểm Silhouette cực tốt hơn so với  $k=3$ ). Thứ hai, quan trọng hơn, là do bản chất của bộ dữ liệu kim cương (trọng lượng và giá cả) là một đại phân phối liên tục, không có các khoảng trống (gap) đứt gãy tự nhiên để tạo thành những "đảo" dữ liệu tách biệt hoàn toàn. Vì vậy, dù chọn k nào thì thuật toán vẫn phải cắt ngang qua những vùng có mật độ dữ liệu dày đặc, làm giảm điểm Silhouette.

3. Nhìn trên hình, cụm có ý nghĩa gì trong lĩnh vực của bạn?

Trả lời: Trong lĩnh vực Thương mại điện tử, 3 cụm này đại diện cho 3 phân khúc sản phẩm kinh doanh cốt lõi dựa trên kích cỡ (carat) và giá trị. Cụm màu xanh dương (carat nhỏ) đại diện cho trang sức phổ thông đại trà; cụm màu xanh lá (carat trung bình) là phân khúc quà tặng/nhân cưới tầm trung; và cụm màu đỏ (carat lớn) là dòng kim cương cao cấp, xa xỉ dành cho giới siêu giàu. Việc chia cụm này giúp sàn thương mại điện tử định hình chiến lược giá và tối ưu hóa hệ thống gợi ý sản phẩm cho từng đối tượng khách hàng khác nhau.

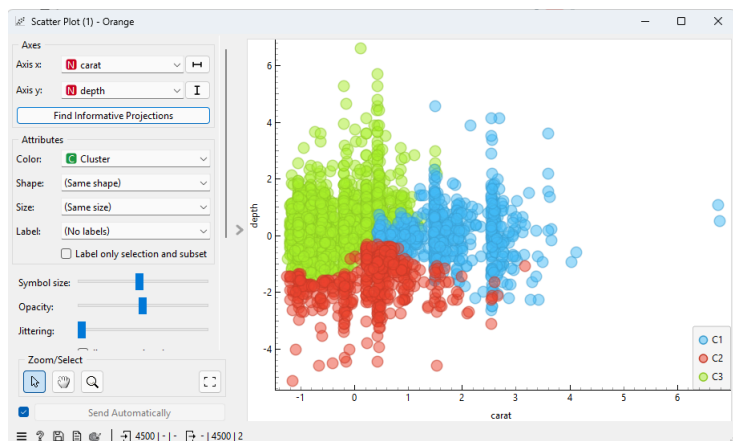
### Bài tập tự làm

Kéo thêm Widget phân cụm Hierarchical Clustering nối chung vào Data gốc (được Distances hóa nếu cần).



• So sánh cụm do nó tạo ra trên Scatter Plot mới so với cụm của k-Means. Xem sự phân bố thành viên cụm có giống nhau không?

Scatter plot mới được tạo ra



2 scatter plot phân bố không giống nhau

Mặc dù cả hai thuật toán đều phân chia dữ liệu thành 3 cụm ( $k = 3$ ), sự phân bố thành viên và hình dáng các cụm có sự khác biệt rõ rệt:

k-Means : Thuật toán có xu hướng chia cắt không gian dữ liệu bằng các "đường ranh giới" tương đối thẳng. Các cụm được hình thành chủ yếu theo trục ngang (carat), tạo ra 3 nhóm khá cân đối về mặt kích thước (khối lượng) từ trái sang phải: cụm xanh dương (nhỏ), cụm xanh lá (trung bình), cụm đỏ (lớn).

Hierarchical Clustering: Thuật toán này lại phân chia dữ liệu phức tạp hơn, có xu hướng bám theo "độ sâu" (depth) nhiều hơn. Cụm xanh lá và cụm đỏ bị chia cắt mạnh theo chiều dọc (trục depth), trong khi cụm xanh dương lại bị kéo dài và bao trọn lấy phần lớn các điểm có carat cao ở phía bên phải.

### ▼ PHẦN 3. GOOGLE COLAB VỚI THƯ VIỆN PYTHON

#### ▼ Import thư viện cần thiết

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

#### ▼ Chọn đặc trưng

Em chọn carat (trọng lượng), price (giá), và depth (tỷ lệ độ sâu) ví:

- Carat và price là hai yếu tố cốt lõi nhất định hình giá trị và phân khúc của một viên kim cương trên sàn thương mại điện tử. depth được thêm vào như một yếu tố chất lượng vật lý ảnh hưởng đến độ bắt sáng, giúp phân rã các cụm theo tinh chất kỹ thuật chứ không chỉ riêng giá tiền.

```
df = sns.load_dataset("diamonds")

# Chọn 3 đặc trưng số: carat (trọng lượng), price (giá), và depth (độ sâu)
features = ['carat', 'price', 'depth']
X_numeric = df[features].copy()
```

#### ▼ Chuẩn hóa dữ liệu

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_numeric)
```

#### ▼ Thử các giá trị k từ 2 đến 6 và lưu lại Inertia cùng Silhouette Score trong một bảng

Dùng random\_state=MSSV trong KMeans để việc lặp lại kết quả có thể kiểm tra được

```
MSSV = 2212453
k_values = range(2, 7)
inertia_values = []
silhouette_scores = []

for k in k_values:
    # Cài đặt mô hình với random_state = MSSV
    model = KMeans(n_clusters=k, random_state=MSSV, n_init=10)
    labels = model.fit_predict(X_scaled)

    # Thu thập Inertia
    inertia_values.append(model.inertia_)

    # Thu thập Silhouette Score (Dùng sample_size=10000 vì dataset diamonds lớn >50k dòng, tính toàn bộ sẽ rất lâu)
    score = silhouette_score(X_scaled, labels, sample_size=10000, random_state=MSSV)
    silhouette_scores.append(score)

# Bảng liệt kê giá trị k, Inertia và Silhouette
results_df = pd.DataFrame({
    'k': k_values,
    'Inertia': inertia_values,
    'Silhouette Score': silhouette_scores
})

print("\nBảng kết quả Inertia và Silhouette Score:")
print(results_df)
```

Bảng kết quả Inertia và Silhouette Score:

| k | Inertia      | Silhouette Score |
|---|--------------|------------------|
| 2 | 91702.440271 | 0.449600         |
| 3 | 69443.823988 | 0.368431         |
| 4 | 51098.421354 | 0.399694         |
| 5 | 42780.040187 | 0.393323         |
| 6 | 37260.497461 | 0.353936         |

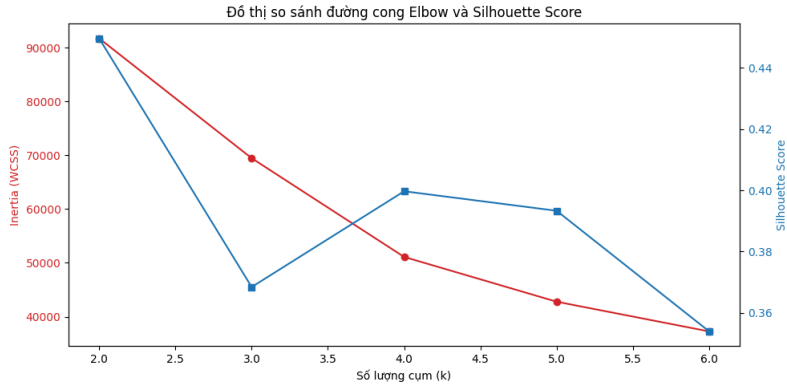
Vẽ ít nhất 2 hình: đồ thị Elbow và biểu đồ cụm cuối cùng trên không gian 2D hoặc PCA 2D (Hình 2 ở bước 6)

```
fig, ax1 = plt.subplots(figsize=(10, 5))

# Đồ thị Elbow (Inertia)
color = 'tab:red'
ax1.set_xlabel('Số lượng cụm (k)')
ax1.set_ylabel('Inertia (WCSS)', color=color)
ax1.plot(k_values, inertia_values, marker='o', color=color, label='Inertia')
ax1.tick_params(axis='y', labelcolor=color)

# Đồ thị Silhouette
ax2 = ax1.twinx()
color = 'tab:blue'
ax2.set_ylabel('Silhouette Score', color=color)
ax2.plot(k_values, silhouette_scores, marker='s', color=color, label='Silhouette')
ax2.tick_params(axis='y', labelcolor=color)

plt.title('Đồ thị so sánh đường cong Elbow và Silhouette Score')
fig.tight_layout()
plt.show()
```



Chọn k cuối cùng dựa trên cả số liệu lẫn bối cảnh nghiệp vụ, không chỉ dựa vào hình đẹp

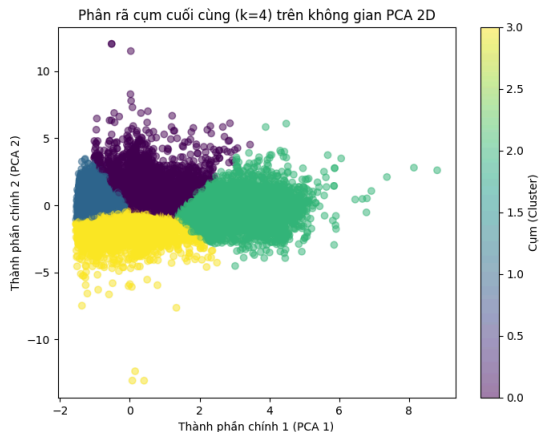
Lý do chọn k = 4

Dựa trên đồ thị, mặc dù Silhouette đạt đỉnh ở k = 2, nhưng lại có một đỉnh cục bộ phức hồi rất rõ nét tại k = 4, đồng thời đường Inertia cũng bắt đầu giảm độ dốc tạo thành điểm gấp tương đối. Hơn nữa, việc chọn k cuối cùng cần dựa trên cả số liệu lẫn bối cảnh nghiệp vụ, do đó chia kim cương trên sàn thương mại điện tử thành 4 phân khúc (phổ thông, tầm trung, cao cấp và siêu cao cấp) sẽ mang lại giá trị thực tiễn cho chiến lược giá và phân tích khách hàng cao hơn nhiều so với việc chỉ chia thành 2 nhóm đơn điệu.

```
k_final = 4
final_model = KMeans(n_clusters=k_final, random_state=MSSV, n_init=10)
final_labels = final_model.fit_predict(X_scaled)

# Dùng PCA giảm về 2 chiều để dễ dàng vẽ biểu đồ cụm trên không gian 2D
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

plt.figure(figsize=(8, 6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=final_labels, cmap='viridis', alpha=0.5)
plt.title(f'Phân rã cụm cuối cùng (k={k_final}) trên không gian PCA 2D')
plt.xlabel('Thành phần chính 1 (PCA 1)')
plt.ylabel('Thành phần chính 2 (PCA 2)')
plt.colorbar(scatter, label='Cụm (Cluster)')
plt.show()
```



Các yêu cầu phải trả lời

1. Elbow và Silhouette có đồng thuận với nhau không?

Elbow và Silhouette không đồng thuận với nhau

Góc độ Silhouette: Tiêu chí này chỉ ra rằng  $k = 2$  là tốt nhất vì đạt điểm số cao nhất (0.449). Tuy nhiên, đường Silhouette lại có một đỉnh phức hợp cực bộ rất đáng chú ý tại  $k = 4$  (đạt 0.399, cao hơn hẳn so với  $k = 3$  và  $k = 5$ ).

Góc độ Elbow (Inertia): Đường cong không tạo thành điểm rõ ràng ở  $k = 2$ . Thay vào đó, điểm gấp xuất hiện ở  $k = 4$ . Từ  $k = 3$  xuống  $k = 4$ , độ chênh lệch Inertia rất lớn (giảm hơn 18.000), nhưng từ  $k = 4$  xuống  $k = 5$ , mức giảm đã chững lại rõ rệt (chỉ còn khoảng 8.000).

2. Nếu hai tiêu chí mâu thuẫn, bạn ưu tiên cách chọn nào và vì sao?

Nên ưu tiên Silhouette Score kết hợp với bối cảnh nghiệp vụ. Inertia luôn có xu hướng giảm khi  $k$  tăng, dễ dẫn đến việc chọn quá nhiều cụm không cần thiết. Silhouette đo lường trực tiếp độ chặt chẽ bên trong cụm và độ tách biệt giữa các cụm, cho thấy chất lượng thực sự của việc gán nhãn.

3. Cụm cuối cùng có thể được diễn giải thành những nhóm đối tượng nào trong lĩnh vực của bạn?

Với lĩnh vực thương mại điện tử, nếu chọn  $k = 3$ , 3 cụm này có thể được diễn giải thành các phân khúc sản phẩm kinh doanh:Cụm 1: Kim cương phổ thông (carat nhỏ, giá thấp) phù hợp trang sức thời trang đại trà.Cụm 2: Kim cương trung cấp, đáp ứng nhu cầu nhân cưới hoặc quà tặng giá trị vừa phải.Cụm 3: Kim cương cao cấp (carat lớn, giá rất cao, chất lượng vật lý xuất sắc) dành cho giới siêu giàu hoặc đầu tư.

Bài tập tự làm

- Viết vòng lặp for tự động thu thập giá trị Inertia và Silhouette Score cho số cụm k chạy từ 2 đến 10.
- Vẽ 2 đồ thị Line Plot riêng biệt: đường cong Elbow (Inertia) và đường Silhouette Score theo k.
- Nếu muốn, sinh viên có thể tích hợp thêm thư viện yellowbrick để dùng KElbowVisualizer và đối chiếu với cách tự động vòng lặp.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from yellowbrick.cluster import KElbowVisualizer

# --- 0. Chuẩn bị dữ liệu ---
df = sns.load_dataset("diamonds")
features = ['carat', 'price', 'depth']
X_scaled = StandardScaler().fit_transform(df[features])

MSSV = 2212453
k_values = range(2, 11) # Chạy k từ 2 đến 10
inertia_values = []
silhouette_scores = []

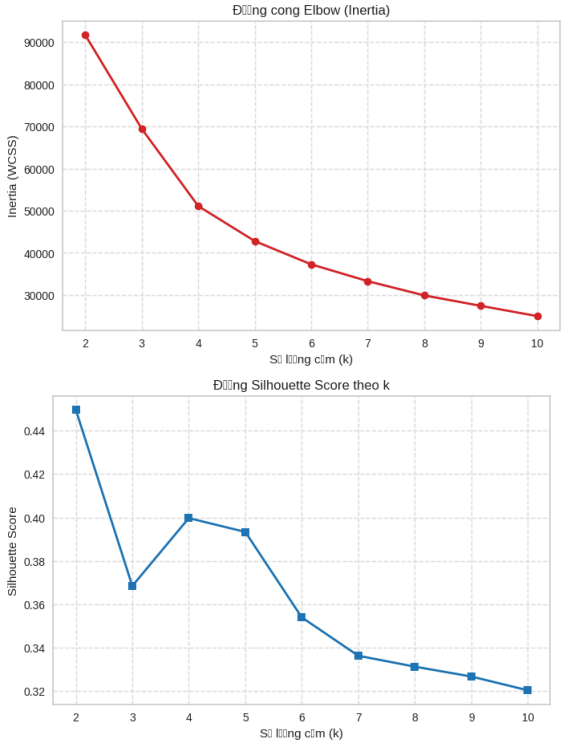
# --- 1. Vòng lặp for tính Inertia và Silhouette ---
for k in k_values:
    model = KMeans(n_clusters=k, random_state=MSSV, n_init=10)
    labels = model.fit_predict(X_scaled)

    inertia_values.append(model.inertia_)
    score = silhouette_score(X_scaled, labels, sample_size=10000, random_state=MSSV)
    silhouette_scores.append(score)
```

```
# --- 2. Vẽ 2 đồ thị Line Plot riêng biệt ---
# Đồ thị 1: Đường cong Elbow (Inertia)
plt.figure(figsize=(8, 5))
plt.plot(k_values, inertia_values, marker='o', color='tab:red', linewidth=2)
plt.title('Đường cong Elbow (Inertia)')
plt.xlabel('Số lượng cụm (k)')
plt.ylabel('Inertia (WCSS)')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

# Đồ thị 2: Đường Silhouette Score
plt.figure(figsize=(8, 5))
plt.plot(k_values, silhouette_scores, marker='s', color='tab:blue', linewidth=2)
plt.title('Đường Silhouette Score theo k')
plt.xlabel('Số lượng cụm (k)')
plt.ylabel('Silhouette Score')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

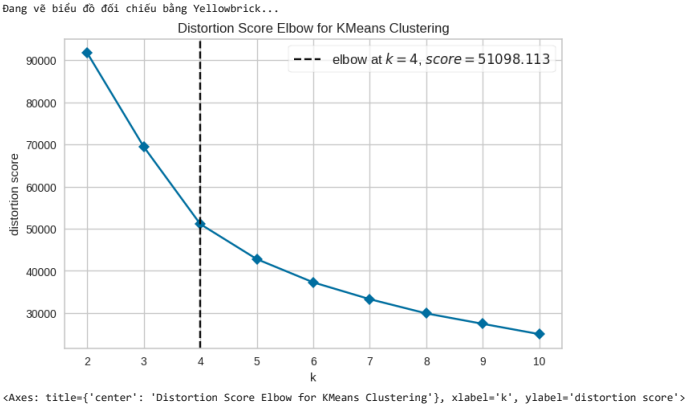
```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 432 (\N{LATIN SMALL LETTER U WITH HORN}) missing from font(s) Liberation Sans.
fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 7901 (\N{LATIN SMALL LETTER O WITH HORN AND GRAVE}) missing from font(s) Liberation Sans.
fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 7889 (\N{LATIN SMALL LETTER O WITH CIRCUMFLEX AND ACUTE}) missing from font(s) Liberation Sans.
fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 7907 (\N{LATIN SMALL LETTER O WITH HORN AND DOT BELOW}) missing from font(s) Liberation Sans.
fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 7909 (\N{LATIN SMALL LETTER U WITH DOT BELOW}) missing from font(s) Liberation Sans.
fig.canvas.print_figure(bytes_io, **kw)
```



```
# --- 3. Tích hợp thư viện Yellowbrick ---
print("Đang vẽ biểu đồ đối chiếu bằng Yellowbrick...")
plt.figure(figsize=(8, 5))
```

```
# Khởi tạo KElbowVisualizer với mô hình KMeans và khoảng k từ 2 đến 10
base_model = KMeans(random_state=MSSV, n_init=10)
visualizer = KElbowVisualizer(base_model, k=(2, 11), timings=False)

# Khớp dữ liệu và hiển thị
visualizer.fit(X_scaled)
visualizer.show()
```



Minh chứng 4–6 dòng kết luận về số cụm hợp lý và ý nghĩa thực tế của cụm

Dựa trên đỉnh cục bộ của Silhouette Score (0.399) kết hợp với điểm gấp của đường cong Inertia,  $k = 4$  được chọn là số cụm hợp lý nhất, thể hiện qua 4 vùng dữ liệu phân tách tương đối rõ ràng trên không gian PCA 2D. Trong lĩnh vực Thương mại điện tử, 4 cụm này mang ý nghĩa phân loại kho kim cương thành 4 phân khúc sản phẩm đặc trưng: phổ thông, tầm trung, cao cấp và siêu cao cấp (hàng sưu tầm). Việc phân tách tự động này giúp ích rất lớn cho các nền tảng bán lẻ trực tuyến trong việc xây dựng hệ thống gợi ý sản phẩm (recommendation system) và chiến lược định giá thông minh. Từ đó, doanh nghiệp có thể triển khai các chiến dịch marketing cá nhân hóa, tiếp cận chính xác tệp khách hàng theo từng mức ngân sách, giúp tối ưu hóa trải nghiệm người dùng và gia tăng tỷ lệ chuyển đổi.